Chrisel Lobo - Core Process & CPU Execution Engine

STC89C52 Microcontroller Multi-Process Simulator - Student Study Guide

Role: Microcontroller Architecture, ISA Specialist & ALU Design

1. What is TCP in Simple Terms?

TCP is the **reliable courier** that carries commands from the UI into your CPU engine.

When the UI wants to execute an instruction, it writes the word `"STEP\n"` into a TCP socket on port 5000. TCP guarantees that your `CoreProcess` receives `"STEP"` without scrambled or missing letters, exactly when sent.

2. Is This the Simplest Connection Method for Beginners?

YES. You don't need to know complex web frameworks or enterprise Java.

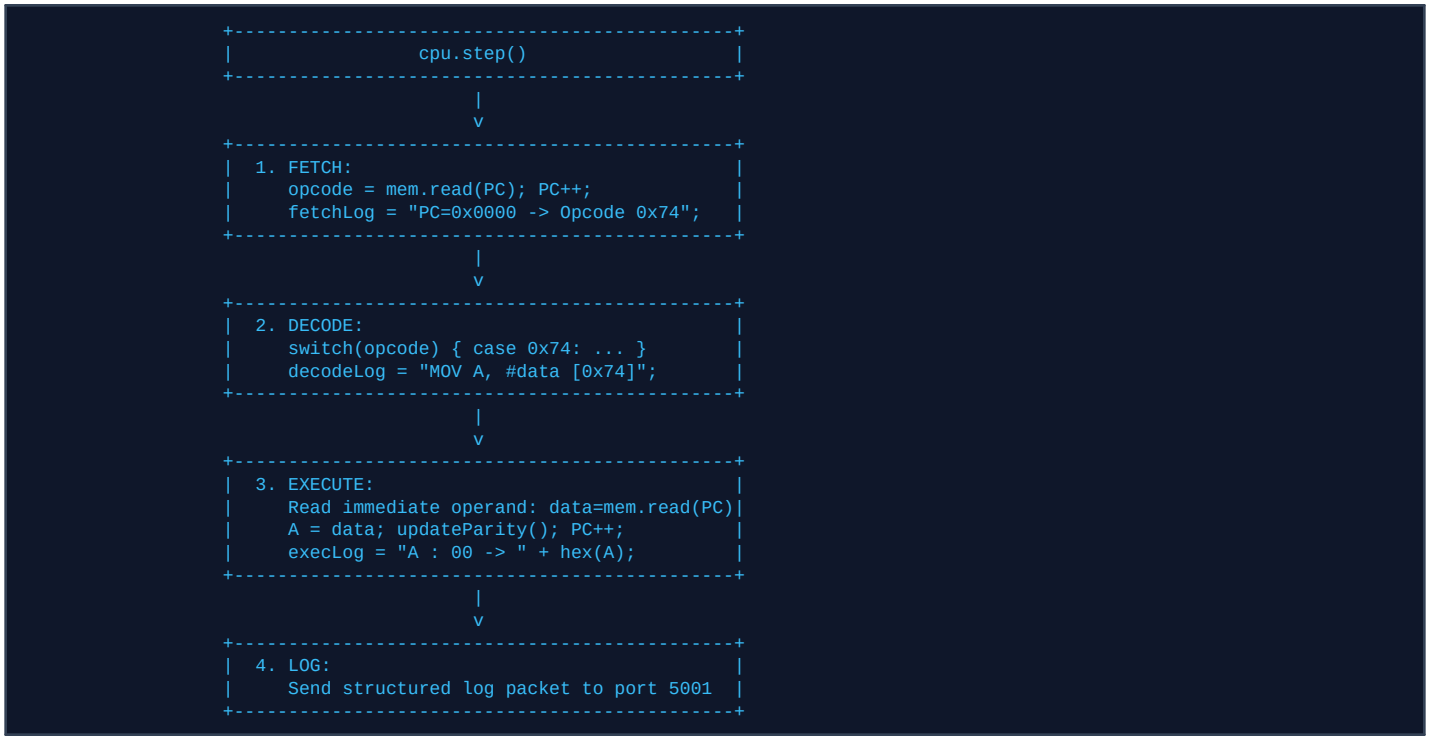
All your `CoreProcess.java` does is:

1. `ServerSocket server = new ServerSocket(5000);` (Wait for someone to connect).
2. `String command = reader.readLine();` (Read "STEP").
3. Call `cpu.step();` (Run the CPU instruction).
4. `writer.println("STATE|PC=0002|A=10...");` (Send the new numbers back).

3. Files Owned & Managed by Chrisel

- `src/CoreProcess.java`: Main server loop on port 5000, socket command dispatcher.
- `src/CPU.java`: 8051 Fetch-Decode-Execute state machine, opcode decoder, ALU flag arithmetic.
- `src/Memory.java`: 256-byte internal RAM model (Register banks, SFRs, scratchpad).
- `src/Registers.java`: Working registers (Accumulator A, B, R0-R7, PC, SP).
- `src/PSW.java`: Program Status Word register (Carry, Aux Carry, Overflow, Parity).
- `src/DataStructures.java`: Hardware Stack Pointer and 10-element Circular FIFO Queue.

4. Flowchart: The CPU Fetch-Decode-Execute Loop (`CPU.java`)



5. Complete Instruction Set Reference

1. `0x00` (`NOP`): No operation. Consumes 1 machine cycle, increments PC.
2. `0x74` (`MOV A, #data`): Immediate move into Accumulator `A`. Updates Parity bit `P`.
3. `0x24` (`ADD A, #data`): Adds immediate byte to Accumulator. Computes `CY`, `AC`, `OV`, `P`.
4. `0x04` (`INC A`): Increments Accumulator `(A + 1) & 0xFF`. Crucial: `INC` does **not** modify `CY`!
5. `0x54` (`ANL A, #data`): Performs bitwise AND `A & data`.
6. `0x80` (`SJMP rel`): Relative jump. Reads signed 8-bit offset `(-128 to +127)` and adds to PC.
7. `0xE0` (`ENQUEUE A`): Inserts value of `A` into 10-element circular queue buffer.
8. `0xD0` (`DEQUEUE A`): Removes oldest byte from queue and loads it into Accumulator `A`.
9. `0xFF` (`HALT`): Sets `isHalt = true`, halting processor execution.

6. Memory Layout & Arithmetic Flags

- **Memory Map (256 Bytes)**:
- `0x00 - 0x1F`: 4 Register Banks of 8 bytes each (Bank 0 is `R0-R7`).
- `0x20 - 0x2F`: 16 Bit-Addressable bytes (128 individual addressable bits).
- `0x30 - 0x7F`: 80 Scratchpad RAM bytes.
- `0x80 - 0xFF`: Special Function Registers (`ACC`, `B`, `PSW`, `SP`).
- **ALU Math Flags in `PSW.java`**:
- **Carry (`CY`)**: Set when addition exceeds 8 bits (`sum > 0xFF`).
- **Auxiliary Carry (`AC`)**: Set when lower nibble carries into bit 4 (`((A & 0x0F) + (data & 0x0F) > 0x0F`). Essential for BCD arithmetic!
- **Overflow (`OV`)**: Signed two's complement overflow. `((A ^ sum) & (data ^ sum) & 0x80) != 0`.
- **Parity (`P`)**: Set to 1 if Accumulator has an odd number of set bits (odd parity rule).

7. How Chrisel Can Add Features as College Teaches More

- **Adding a New Instruction (e.g. `CPL A` - Complement Accumulator, Opcode `0xF4`):**

Open `src/CPU.java`, find the `switch(opcode)` statement, and add 4 lines:

```
case (byte) 0xF4: // CPL A (invert all 8 bits)
    currInstrcName = "CPL A";
    regs.A = (~regs.A) & 0xFF;
    updateParity();
    break;
```

- **Adding a New Register (e.g. `DPTR` - 16-bit Data Pointer):**

Open `src/Registers.java`, add `public int DPTR = 0x0000;`, and in `CoreProcess.java` emit `|DPTR=` in `emitState()`.

---

8. Chrisel's 5 Hostile Examiner Questions & Defenses

Q1: "Why does the Stack Pointer (SP) reset to 0x07 instead of 0x00?"

Answer: *"Internal RAM addresses 0x00 through 0x07 are occupied by Register Bank 0 (registers R0 to R7). In the 8051 architecture, a PUSH operation executes as a pre-increment: ++SP. By resetting SP to 0x07, the very first pushed value is stored at address 0x08, safely above Register Bank 0."*

Q2: "Why do you use bitwise masking `& 0xFF` on all register math?"

Answer: *"In Java, primitive `byte` is signed (-128 to 127) and standard integer arithmetic promotes to 32-bit signed ints. The STC89C52 is an 8-bit unsigned hardware microcontroller. Masking with `& 0xFF` truncates the 32-bit integer result to an exact 8-bit unsigned value in the range 0 to 255."*

Q3: "How does your SJMP instruction handle backward loops?"

Answer: *"The relative offset byte in memory is cast to a signed Java byte: `byte offset = (byte) mem.read(PC)`. When offset is negative (e.g. `0xFE` = -2), Java sign-extends it, subtracting from PC and allowing backward jump loops without extra logic."*

Q4: "Why did you implement a Circular Queue instead of an ArrayList?"

Answer: *"In real-time embedded systems, dynamic heap allocation via ArrayList causes unpredictable garbage collection pauses and memory fragmentation. A circular array uses fixed memory pre-allocated at startup and provides guaranteed O(1) enqueue and dequeue operations."*

Q5: "How does your simulator calculate the Parity (P) bit?"

Answer: *"The 8051 Parity bit reflects odd parity of Accumulator A. We count set bits using `Integer.bitCount(A)`. If the number of 1-bits is odd, P is set to 1; if even, P is cleared to 0."*